Constructive Reflection Beyond Contender 5

An axiom-free Lean entry with accessible forests,
structural tree recursion, and unbounded finite-type uniformization

Research prepared for Vladimir Reshetnikov

12 October 2026 | Lean 4.19.0

Abstract

We construct a closed, executable Lean natural number and prove that it is strictly larger than Contender 5 in Cody Roux's *name-the-biggest-number* repository. The comparison uses a source-faithful Lean port of the original total raw-term interpreter, including its treatment of ill-typed terms. The new construction has three substantive ingredients: bounded evaluation in a language with the full recursor for countably branching trees; a reflection operator with an accessible countable family of function oracles; and a structurally generated tree schedule that diagonalizes over an unbounded dependent family of finite functional types. Every numerical definition is constructive. The delivered comparison, enumeration, and diagonalization certificates have empty Lean axiom dependency lists.

The strength statements are explicit. Reflection eventually exceeds every fixed program in its specified oracle language; successor stages escape the preceding stage's language; accessible forest stages preserve the exact values of all their child oracles. We also give finite-path growth theorems, an independently checked earlier contender that the final entry exceeds, and a precise explanation of why the construction neither invokes a Busy Beaver oracle nor defines a universal evaluator for all Lean programs. We make no claim of an optimal entry under an unspecified machine budget, and we do not assign an unproved named ordinal to the raw tree schedule.

Deliverable. A complete Lean source package, a compiled article, reproducible checks, original-source provenance, and the theorem `Original.contender5` $<$ `contender_6`. The final numeral is specified and compared, not expanded in decimal.

Contents

1 The result and the adapted contest **3**

2 Auditing the exact baseline **4**
2.1 Simple types and the total raw-term interpreter 4
2.2 Depth and maximum selection 4
2.3 What the port certificate does and does not mean 5

3 A general bounded reflection principle **5**

4 Reflection with full countably branching tree recursion **6**
4.1 The inductive tree type 6
4.2 The object language $\mathcal{W}_f$ 6
4.3 The unary tree-language reflector 7

5 Why a maximum of child values is not an accessible forest **8**
5.1 Forest syntax and semantics 8

6 A large, explicit structural schedule **9**
6.1 Tree arithmetic and Veblen-style operations 9
6.2 Uniformization over unbounded finite functional levels 9

7 Two hierarchies and the final entry **10**
7.1 The unary reflection hierarchy 10
7.2 The accessible forest hierarchy 11
7.3 The final oracle forest and natural number 12

8 Limits of the strength claims **12**
8.1 Raw trees are not canonical ordinals 12
8.2 Oracle order and input order 13
8.3 A function theorem does not rank every fixed integer 13
8.4 Constructive totality and the absence of Busy Beaver search 13

9 Formal verification and reproducibility **13**
9.1 Axiom discipline 13
9.2 Why typechecking remains short 14
9.3 Observed build and regression checks 14

10 Further research questions **15**

11 Conclusion **16**

A Guide to the source modules **17**

1 The result and the adapted contest

The entry is a natural number N, given by a closed Lean definition named `contender_6`. Its principal certificate is

$$\boxed{C_5 < N.}$$

Here C_5 is the exact numerical interpretation of the repository's fifth contender, ported to Lean as `Original.contender5`. The word "exact" refers to the computational equations: the same raw syntax, enumeration, reverse-index lookup, paired casts, default values, and maximum-selection algorithm are retained. Section 2 states the cross-language verification boundary precisely.

The repository asks for constructive definitions, no added axioms, and a formal strict comparison with the previous contender. It specifies 15 seconds for checking a definition and one minute for its comparison proof, on a reasonable machine. It discourages defining an entry by applying a trivial operation to the previous contender's named constant [1]. We retain those intentions and substitute Lean for Coq.

Requirement	Lean version used here
Underlying object	A closed term of type `Nat`.
Constructivity	Ordinary total definitions and structural recursion; no `noncomputable`, unbounded halting oracle, or partial numerical definition.
Axiom policy	The entry and the comparison theorem must each print an empty axiom dependency list. The supplied audit also checks the supporting theorems.
Checking budget	Retain the original 15-second and 60-second targets; report observed times and the pinned toolchain.
Previous entry	Compare with a faithful port of the repository's value, including malformed raw programs.
Use of prior work	Reuse the general interpreter and a semantic embedding, not the named `contender5` constant in the new numerical definitions.
Submission	A reviewable archive, without publishing a pull request on the user's behalf.

The README gives checking deadlines, not a separately specified research or typing deadline. Thus "the largest number in the allotted time" is treated as a request to construct a strong, fully checked entry during this work. It does not define a mathematical optimization problem with a fixed machine, source encoding, and admissible set of programs. With all those parameters fixed and a hard total checking-time cap, only finitely many source strings may be readable and an admissible maximum could exist. We make no claim to have found it. The familiar observation that $m + 1 > m$ establishes the absence of a largest natural number; it does not by itself establish the absence of a maximum under a fixed finite resource cap.

The useful research question is consequently structural: can a short, transparent, total definition make an expressive-strength advance over the existing entry, with a comparison certificate that stays small? The answer here is yes. The final construction improves a fully verified intermediate entry and exposes an entire forest of previous evaluators to the next object language. This latter step is more informative than merely making a larger finite maximum of their numerical outputs.

2 Auditing the exact baseline

The inspected repository revision is

`d56dce013e6264292533efdc5e1de71863c6fb86`.

The archive retains the upstream files and their license. The computational part of `Contender.v` is translated in `Original.lean`; the original comparison with Contender 4 is not needed to prove the new result.

2.1 Simple types and the total raw-term interpreter

The original type language is

$$\sigma, \tau ::= \mathbb{N} \mid \sigma \to \tau, \qquad [\![\mathbb{N}]\!] = \mathbb{N}, \quad [\![\sigma \to \tau]\!] = [\![\sigma]\!] \to [\![\tau]\!].$$

Terms have variables, annotated lambdas, application, zero, successor, and a natural-number recursor at each simple result type. Environments contain dependent pairs (σ, v) with $v : [\![\sigma]\!]$. A lambda returns a native total Lean function. The interpreter is structural recursion on syntax, even though applying one of the resulting higher-type values can initiate a very long computation.

Two details matter for an exact comparison. First, the repository's variables use levels measured from the old end of the environment. If the environment has length ℓ, a variable i looks up the list position $\ell - (i + 1)$ when $i < \ell$, and returns an error value otherwise. Prepending a newly bound value therefore does not change the meaning of the old variable levels. Second, casts between unequal types return recursively defined defaults. At a function type, the default is a constant function returning the codomain default. The paired arrow casts convert arguments in one direction and results in the other. These conventions are preserved.

In particular, the enumeration is *not* filtered by an external typing judgment. All raw terms are interpreted. At top level, a numerical result is returned as a natural number, while a function result contributes zero. A proof about a maximum over well-typed closed terms alone would not identify the target in this repository. The Lean port avoids that substitution.

2.2 Depth and maximum selection

Let d_T be type depth and d_0 original term depth. They satisfy

$$\begin{aligned} d_T(\mathbb{N}) &= 1, & d_T(\sigma \to \tau) &= 1 + \max(d_T(\sigma), d_T(\tau)), \\ d_0(\mathsf{var}\ i) &= i + 2, & d_0(\mathsf{app}\ u\ v) &= 1 + \max(d_0(u), d_0(v)), \\ d_0(\lambda_{\sigma,\tau} u) &= 1 + \max(d_T(\sigma), d_T(\tau), d_0(u)). \end{aligned}$$

Zero and successor have depth one, and a recursor annotated by ρ has depth $1 + d_T(\rho)$. Let $L_0(d)$ be the exact list generated by the source at bound d, and let $e_0(t)$ be its numerical evaluation.

The source selects a maximizing term with an accumulator `maxBy`. For proofs it is convenient to use the value-only accumulator

$$\mathrm{foldMax}(g, c, []) = c, \qquad \mathrm{foldMax}(g, c, x :: xs) = \mathrm{foldMax}(g, \max(c, g(x)), xs).$$

Lemma 2.1 (Maximum-selection invariant)**.** *If $g(b) = c$, then the source's maximum-selection routine returns a term whose value is* $\mathrm{foldMax}(g, c, xs)$.

Proof. Induct on the list. The empty case is the accumulator invariant. In the step case, if $c < g(x)$, both procedures replace their numerical accumulator by $g(x)$, and the term accumulator becomes x. Otherwise both numerical accumulators stay c, and the current best term is retained. The induction hypothesis applies in either branch. □

Consequently, with the empty maximum equal to zero,

$$M_0(d) = \max(\{0\} \cup \{e_0(t) : t \in L_0(d)\}), \qquad C_5 = M_0(42).$$

This equality is proved in Lean, rather than replacing the source definition silently. Both directions of $t \in L_0(d) \iff d_0(t) \le d$ are also checked.

2.3 What the port certificate does and does not mean

Lean checks all subsequent comparisons against the translated definition. The correspondence with the Coq source is a constructor-by-constructor source audit, not a theorem produced by a verified Coq-to-Lean compiler. That boundary is recorded in the archive. It is made reviewable by keeping the original source, pinning its revision, retaining `maxBy`, and testing the delicate lookup, binding, casting, and malformed-term cases. The small values $M_0(d)$ for $d = 0, 1, 2, 3, 4$ are respectively $0, 0, 1, 2, 3$. No attempt is made to compute $M_0(42)$.

3 A general bounded reflection principle

The central method does not require an ordinal analysis. It needs a total interpreter, a finite syntax enumeration, and an explicit running input. The running input is essential for the strongest conclusion: a maximum over closed descriptions measures description size, whereas a language with an input constant supports an eventual comparison between functions.

Fix a family of oracles $\mathcal{O}$, a numerical interpreter $e_{\mathcal{O},n}(t)$, and finite lists $L(d)$ satisfying completeness:

$$\operatorname{depth}(t) \le d \quad \Longrightarrow \quad t \in L(d).$$

For an explicit positive overhead $p(n)$, consider

$$B_{\mathcal{O}}(n) = p(n) + \max\bigl(A_{\mathcal{O}}(n), \max_{t \in L(n+3)} e_{\mathcal{O},n}(t)\bigr), \qquad p(n) \ge 1.$$

The auxiliary term A can provide useful uniform inequalities; it does not compromise totality.

Theorem 3.1 (Bounded and eventual diagonalization)**.** *For each term t,*

$$\operatorname{depth}(t) \le n + 3 \quad \Longrightarrow \quad e_{\mathcal{O},n}(t) < B_{\mathcal{O}}(n).$$

In particular, the strict inequality holds for every $n \ge \operatorname{depth}(t)$. No fixed term t represents the whole function $B_{\mathcal{O}}$ in this same oracle language.

Proof. Completeness places t in the finite maximum. Its value is therefore no larger than that maximum, and adding the positive overhead makes the result strictly larger. For nonrepresentability, suppose that $e_{\mathcal{O},n}(t) = B_{\mathcal{O}}(n)$ for every n, and use $n = \operatorname{depth}(t)$. This yields a number strictly smaller than itself. □

This theorem is stronger than a comparison with one selected previous integer. It supplies a uniform escape from every fixed program of a specified earlier language. It is also sharply scoped: "every fixed program" means a single finite expression interpreted in that language, not every Lean function. The meta-level interpreter uses dependent types to interpret all

simple types uniformly. The object language does not thereby acquire a total evaluator for its own entire meta-language.

The intermediate `Reflection.lean` module implements this principle for the original natural-number language with a unary oracle and an input constant. Its purpose is to isolate and check the idea before adding tree recursion. The final entry uses the richer versions below.

4 Reflection with full countably branching tree recursion

4.1 The inductive tree type

Introduce the strictly positive inductive type

$$\mathsf{B} ::= \mathsf{Z} \mid \mathsf{S}(a) \mid \mathsf{L}(f), \qquad a : \mathsf{B}, \quad f : \mathbb{N} \to \mathsf{B}.$$

These are raw well-founded, countably branching trees. A closed tree is described by a finite program that constructs its branches on demand. It need not be a finite combinatorial tree. Lean accepts structural recursion because every $f(i)$ is a recursive child of $\mathsf{L}(f)$.

The recursor with result type X has parameters

$$z : X, \quad s : \mathsf{B} \to X \to X, \quad \ell : (\mathbb{N} \to \mathsf{B}) \to (\mathbb{N} \to X) \to X$$

and equations

$$\begin{aligned} \mathrm{rec}_{\mathsf{B}}(z, s, \ell, \mathsf{Z}) &= z, \\ \mathrm{rec}_{\mathsf{B}}(z, s, \ell, \mathsf{S}(a)) &= s\big(a, \mathrm{rec}_{\mathsf{B}}(z, s, \ell, a)\big), \\ \mathrm{rec}_{\mathsf{B}}(z, s, \ell, \mathsf{L}(f)) &= \ell\big(f, i \mapsto \mathrm{rec}_{\mathsf{B}}(z, s, \ell, f(i))\big). \end{aligned}$$

Thus both the original subtree and its recursive result are available in a successor branch, and both whole families are available in a limit branch. This is the full primitive recursor, not just a scalar traversal.

4.2 The object language $\mathcal{W}_f$

Its types are generated by $\mathbb{N}$, B, and arrows. In addition to variables, annotated lambdas, and application, the language contains the following constants. The result type ρ ranges over *all* its finite types.

Constant	Type or meaning
$0, \mathsf{succ}$	$\mathbb{N}$, $\mathbb{N} \to \mathbb{N}$.
nrec_ρ	$\rho \to (\mathbb{N} \to \rho \to \rho) \to \mathbb{N} \to \rho$.
$\mathsf{Z}, \mathsf{S}, \mathsf{L}$	B, $\mathsf{B} \to \mathsf{B}$, $(\mathbb{N} \to \mathsf{B}) \to \mathsf{B}$.
brec_ρ	$\rho \to (\mathsf{B} \to \rho \to \rho) \to ((\mathbb{N} \to \mathsf{B}) \to (\mathbb{N} \to \rho) \to \rho) \to \mathsf{B} \to \rho$.
O, X	$\mathbb{N} \to \mathbb{N}$, $\mathbb{N}$, interpreted by f and the current input n.
$\mathsf{old}(t)$	A natural-number constant with value $e_0(t)$, for any raw original term t.

The old constructor is an explicit closed numerical embedding. It does not reinterpret the old term under the new environment. It therefore preserves exactly the observable value relevant to the contest, including all prescribed malformed-term results; nonnumeric top-level results contribute zero. A malformed term need not contribute zero: applying successor to successor contributes one under the source's error-cast rules. This avoids an unnecessary identification between the two different semantic type families. The new language independently has all the usual simple-type constructors and its own variables and recursors.

As in the baseline, every raw term has a total interpretation. The default at B is Z; other defaults and paired arrow casts are defined structurally. At top level, a tree or function result contributes zero. This is a semantics of all raw expressions, not an assertion that all expressions are conventionally well typed.

Let $L_W(d)$ be the finite enumeration and let $e^W_{f,n}$ be its numerical interpreter. The tree primitives, oracle, and input have depth one. A recursor annotated by ρ has depth $1 + d_T(\rho)$, and $\mathrm{depth}(\mathsf{old}(t)) = d_0(t)$. The remaining depth rules are the familiar maximum rules for applications and annotated lambdas.

Theorem 4.1 (Exact enumeration and totality)**.**

$$t \in L_W(d) \quad \Longleftrightarrow \quad \mathrm{depth}(t) \le d.$$

The evaluator, enumeration, and numerical maxima are total constructive definitions.

Proof. The forward and backward implications are simultaneous elementary inductions on the depth bound and the appropriate syntax constructors. A lambda uses two types and a smaller body; an application uses two smaller terms; a recursor uses a smaller annotation. The old branch uses the previously proved original completeness and soundness. The finite lists are built by finite products, maps, and concatenations.

The interpreter recurses structurally on syntax and on type annotations for casts. Natural and tree recursors are already total recursors of inductive types. In particular, evaluation at a function type produces an actual total function, and a tree-recursion branch receives a family of already defined recursive values. No search for a normalization certificate is performed. □

4.3 The unary tree-language reflector

Define

$$\mathcal{R}(f)(n) = n + 1 + \max\left(f(n), \max_{t \in L_W(n+3)} e^W_{f,n}(t)\right). \tag{1}$$

It is `Arboreal.Rw` in the source.

Theorem 4.2 (Uniform lower bounds and language escape)**.** *For every f, n,*

$$f(n) < \mathcal{R}(f)(n), \qquad n < \mathcal{R}(f)(n).$$

Whenever $d \le n + 3$,

$$M_0(d) < \mathcal{R}(f)(n).$$

For every fixed t in $\mathcal{W}_f$,

$$n \ge \mathrm{depth}(t) \quad \Longrightarrow \quad e^W_{f,n}(t) < \mathcal{R}(f)(n).$$

In particular $C_5 < \mathcal{R}(f)(42)$, and no fixed $\mathcal{W}_f$ expression represents $\mathcal{R}(f)$.

Proof. The first two inequalities follow directly from (1). Each old term of depth at most d occurs as $\mathsf{old}(t)$ in the larger enumeration, with exactly the same numerical value. Thus the whole old maximum is bounded by the new maximum. The positive overhead makes the inequality strict. The last statements follow from Section 3. □

5 Why a maximum of child values is not an accessible forest

A pointwise maximum can majorize a collection of functions while concealing the individual values needed to execute their programs. For example, knowing $\max(f(n), g(n))$ need not reveal whether $f(n) = 0$. An object program can branch on such a test. We therefore distinguish two operations carefully.

The first hierarchy below uses the finite join $\max_{i \le n} f_i(n)$. It certifies eventual numeric domination of each fixed child, but that alone does not establish that the next stage can simulate every child-oracle program. The final construction additionally uses *accessible forests*: a new language receives the full function $\Phi : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$, so it can ask for the exact value $\Phi(i, m)$.

5.1 Forest syntax and semantics

The forest language has a base constructor containing any term of $\mathcal{W}$, interpreted with unary oracle $\Phi(0, \cdot)$ and with the current input. It also has indexed oracle constants $\mathsf{O}_i : \mathbb{N} \to \mathbb{N}$, a family constant $\mathsf{Family} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$, applications, and annotated lambdas. The indexed oracle denotes $\Phi(i, \cdot)$; the family constant denotes Φ itself. This last primitive permits the selected child index to depend on the running input, rather than requiring a fixed index written into syntax.

The base constructor preserves the full environment. Consequently the forest language retains the tree and natural-number recursors at every simple result type, and can combine them with calls to any of its child oracles. An indexed oracle has depth $i + 1$, the family constant has depth one, and base terms retain their $\mathcal{W}$-depth. The enumerator is finite at every bound: its indexed oracle branch ranges over only finitely many indices. Write its list as $L_F(d)$ and its numerical semantics as $e^F_{\Phi,n}$.

Define the prefix maximum

$$P_\Phi(n) = \max_{0 \le i \le n} \Phi(i, n)$$

and the forest reflector

$$\mathcal{F}(\Phi)(n) = n + 2 + \max\left(P_\Phi(n), \max_{t \in L_F(n+3)} e^F_{\Phi,n}(t)\right). \tag{2}$$

The extra unit relative to (1) makes a useful comparison strict.

Theorem 5.1 (Accessible forest reflection)**.** *For $i \le n$,*

$$\Phi(i, n) < \mathcal{F}(\Phi)(n).$$

For each fixed forest expression t,

$$n \ge \mathrm{depth}(t) \quad \Longrightarrow \quad e^F_{\Phi,n}(t) < \mathcal{F}(\Phi)(n).$$

Also

$$\mathcal{R}(\Phi(0, \cdot))(n) < \mathcal{F}(\Phi)(n).$$

No fixed expression in the language with this forest oracle represents the whole function $\mathcal{F}(\Phi)$.

Proof. The prefix maximum contains $\Phi(i, n)$. Completeness contains every expression of sufficiently small depth. The respective positive overheads give the first two claims. For the comparison with $\mathcal{R}$, every $\mathcal{W}$-term occurs as a forest base term at the same depth, and $P_\Phi(n) \ge \Phi(0, n)$. Thus the maximum in (2) is at least the maximum in (1); the overhead is larger by one. Nonrepresentability again uses the depth of a proposed representing expression as the diagonal input. □

The theorem quantifies over the actual forest-language semantics. A program that combines finitely many child functions, or selects a child uniformly through Family, is beaten once its finite description fits the depth budget. This is the precise sense in which the forest construction repairs the information loss of a numerical maximum. It does not assert a reflection principle for arbitrary formal theories or for all meta-level Lean definitions.

The checked escape theorem concerns the forest syntax just specified. The observation that the primitives permit a syntax-directed translation of individual child-oracle programs is not presented as a separately machine-checked generic translation theorem; quantitative verified translations remain a proposed next step.

6 A large, explicit structural schedule

Both reflection operators can already produce large integers. We now give an explicit schedule for iterating them that uses more than a fixed finite height. All definitions in this section construct raw B-trees. Their correctness as total programs is independent of any assignment of ordinal names.

6.1 Tree arithmetic and Veblen-style operations

For any type X, write

$$\mathrm{fold}(z, s, \ell, \mathsf{Z}) = z, \quad \mathrm{fold}(z, s, \ell, \mathsf{S}(a)) = s(\mathrm{fold}(z, s, \ell, a)),$$

with the analogous limit equation $\ell(i \mapsto \mathrm{fold}(z, s, \ell, f(i)))$. Define iteration by $\mathrm{it}(u, 0, x) = x$ and $\mathrm{it}(u, k+1, x) = \mathrm{it}(u, k, u(x))$. Let

$$\underline{k} = \mathrm{it}(\mathsf{S}, k, \mathsf{Z}), \qquad \omega_{\mathsf{B}} = \mathsf{L}(k \mapsto \underline{k}).$$

The structural arithmetic operations are

$$\begin{aligned} a + b &= \mathrm{fold}(a, \mathsf{S}, \mathsf{L}, b), \\ a \cdot b &= \mathrm{fold}(\mathsf{Z}, r \mapsto r + a, \mathsf{L}, b), \\ a^b &= \mathrm{fold}(\mathsf{S}(\mathsf{Z}), r \mapsto r \cdot a, \mathsf{L}, b). \end{aligned}$$

Put $p(a) = \omega_{\mathsf{B}}^a$. For a tree transformation u, define

$$\begin{aligned} \mathrm{cl}(u, a) &= \mathsf{L}(k \mapsto \mathrm{it}(u, k, \mathsf{S}(a))), \\ \mathrm{der}(u) &= \mathrm{fold}(\mathrm{cl}(u, \mathsf{Z}), a \mapsto \mathrm{cl}(u, a), \mathsf{L}), \\ \mathrm{common}(u_0, u_1, \ldots) &= \mathrm{der}\big(a \mapsto \mathsf{L}(k \mapsto u_k(a))\big), \\ V_u &= \mathrm{fold}(u, \mathrm{der}, \mathrm{common}). \end{aligned}$$

Thus $V_u(a)$ is a tree transformation and V_p is a Veblen-style schedule. These equations are the specification, not an appeal to an informally understood large ordinal. Their names explain the construction's inspiration. Establishing equality with a particular canonical Veblen notation system would require additional invariants and a separate proof.

6.2 Uniformization over unbounded finite functional levels

A more consequential step is to organize all finite levels of higher-order transformations in a dependent family:

$$E_0 = \mathsf{B} \to \mathsf{B}, \qquad E_{k+1} = E_k \to E_k.$$

This is an actual type-valued recursive definition in Lean. At level zero, use common for countable limits and the derivative of a diagonal evaluation for tree-indexed diagonals. Lift these operations pointwise:

$$\begin{aligned} \Lambda_0(u_i) &= \mathrm{common}(u_i), & \Delta_0(v_a) &= \mathrm{der}(a \mapsto v_a(\mathsf{Z})), \\ \Lambda_{k+1}(u_i)(x) &= \Lambda_k(i \mapsto u_i(x)), & \Delta_{k+1}(v_a)(x) &= \Delta_k(a \mapsto v_a(x)). \end{aligned}$$

For $s : E_{k+1}$ and $z : E_k$, define

$$Q_k(s)(z) = \Delta_k\big(a \mapsto \mathrm{fold}(z, s, \Lambda_k, a)\big).$$

Then $Q_k : E_{k+2}$. Choose canonical seeds

$$q_0 = p, \qquad q_1 = \mathrm{der}, \qquad q_{k+2} = Q_k.$$

The operation $\mathrm{collapse}_k : E_k \to E_0$ applies a functional to the canonical seed one level below until level zero is reached:

$$\mathrm{collapse}_0(u) = u, \qquad \mathrm{collapse}_{k+1}(u) = \mathrm{collapse}_k(u(q_k)).$$

Two useful checks reduce by definition:

$$\begin{aligned} \mathrm{collapse}_1(q_1) &= \mathrm{der}(p), \\ \mathrm{collapse}_2(q_2) &= \mathrm{der}(a \mapsto V_p(a)(\mathsf{Z})). \end{aligned}$$

These identities are checked by `rfl` in Lean.

Finally set

$$U(a) = \mathsf{L}\big(k \mapsto \mathrm{collapse}_{k+2}(q_{k+2})(\mathsf{S}(a))\big), \tag{3}$$

$$\Theta = \mathsf{L}\big(m \mapsto \mathrm{it}(U, m+1, \mathsf{Z})\big). \tag{4}$$

This Θ is `treeIndex`. Equation (3) ranges over unbounded finite functional levels in one uniformly typed definition; equation (4) then iterates that operation. The integer 42 does not appear in this construction.

Dependent uniformization of this kind is motivated by the distinction between fixed simple types and a universe containing a type family indexed by natural numbers. Escardó gives an explicit development of related Church encodings and Brouwer trees [2]. We use that perspective to design a schedule; the delivered definitions and proofs stand on their own. No exercise postulate from that development is imported.

7 Two hierarchies and the final entry

7.1 The unary reflection hierarchy

Define functions $H_a : \mathbb{N} \to \mathbb{N}$ by structural recursion on $a : \mathsf{B}$:

$$\begin{aligned} H_{\mathsf{Z}} &= \mathcal{R}(\mathsf{succ}), \\ H_{\mathsf{S}(a)} &= \mathcal{R}(H_a), \\ H_{\mathsf{L}(c)} &= \mathcal{R}\big(n \mapsto \max_{i \le n} H_{c(i)}(n)\big). \end{aligned} \tag{5}$$

Every call in the limit branch is on an immediate recursive child. The argument n may vary, but the structurally decreasing argument is the tree. This is a standard higher-type use of the recursor.

Theorem 7.1 (Successors, limits, and bounded paths)**.** *For every a, n, $H_a(n) < H_{\mathsf{S}(a)}(n)$. For $i \le n$,*

$$H_{c(i)}(n) < H_{\mathsf{L}(c)}(n).$$

If a path of r upward steps uses successor edges or limit edges whose selected indices are at most n, then

$$H_a(n) + r \le H_b(n)$$

for its starting and ending trees a, b.

Proof. Successor strictness is $f(n) < \mathcal{R}(f)(n)$. At a limit, the selected child is bounded by the prefix maximum, which is strictly below its reflection. Induction on the finite path adds one to the certified gap at every step. No monotonicity assumption on the reflector's oracle argument is used. □

The same outer-constructor argument proves

$$C_5 < H_a(42) \quad \text{for every } a : \mathsf{B}.$$

For this statement it is not necessary to traverse the potentially enormous tree: each of its three possible outer constructors already applies $\mathcal{R}$.

Define $\mathrm{pad}(a, 0) = a$ and $\mathrm{pad}(a, k+1) = \mathsf{S}(\mathrm{pad}(a, k))$, and put

$$C = \mathsf{L}(k \mapsto \mathrm{pad}(\Theta, k+1)), \qquad B(n) = H_C(n). \tag{6}$$

Padding is literal iteration of $\mathcal{R}$ at the function level:

$$H_{\mathrm{pad}(a,k)} = \mathcal{R}^{[k]}(H_a).$$

This function equality is established by structural induction and congruence; function extensionality is unnecessary.

Corollary 7.2 (Diagonalization in reflection height)**.**

$$\mathcal{R}^{[n+1]}(H_\Theta)(n) < B(n).$$

In particular $\mathcal{R}^{[43]}(H_\Theta)(42) < B(42)$, and $C_5 < B(42)$.

Proof. Select child $i = n$ in (6) and use the padding identity. □

7.2 The accessible forest hierarchy

Write $\overline{f}$ for the constant family $i \mapsto f$. Define

$$\begin{aligned} K_{\mathsf{Z}} &= \mathcal{F}(\overline{\mathsf{succ}}), \\ K_{\mathsf{S}(a)} &= \mathcal{F}(\overline{K_a}), \\ K_{\mathsf{L}(c)} &= \mathcal{F}(i \mapsto K_{c(i)}). \end{aligned} \tag{7}$$

The crucial distinction from (5) is that the limit reflector receives the exact child family, not just its maximum at the current input. The family primitive can call any $K_{c(i)}(m)$ requested by a finite program.

For every $i \le n$, the strict child inequality $K_{c(i)}(n) < K_{\mathsf{L}(c)}(n)$ follows from forest reflection. At a successor, the corresponding forest program can use the preceding function uniformly, and the next stage eventually exceeds every fixed expression in that language. At a limit, it can also use uniformly selected children. These claims follow from the same exact finite-depth theorem rather than from an assumed monotonicity principle.

7.3 The final oracle forest and natural number

The final entry strictly improves the completed unary candidate as well as using the accessible-forest hierarchy. Define the family Ψ by

$$\Psi(0, n) = B(n), \qquad \Psi(k+1, n) = K_{\mathrm{pad}(C,k)}(n). \tag{8}$$

Our proposed contest number is

$$\boxed{N = \mathcal{F}(\Psi)(42).} \tag{9}$$

Every component in (9) has been specified by finite syntax and total structural recursion. In particular, Ψ is not a list of arbitrary unprovided functions. It is one closed program producing the appropriate function for any requested index.

Theorem 7.3 (Final comparison and explicit structural margins)**.** *The final entry satisfies*

$$\begin{aligned} C_5 &< B(42) < N, \\ \mathcal{R}^{[43]}(H_\Theta)(42) &< B(42) < N, \\ K_{\mathrm{pad}(C,41)}(42) &< N. \end{aligned}$$

The last expression is 41 additional successor forest-reflections above K_C. Moreover $K_C(42)$ itself strictly exceeds the child with 43 successor forest-reflections above K_Θ.

Proof. The first chain uses the proved unary candidate inequality and oracle zero in (8). The second is its reflection-height lower bound. For the third, select oracle index 42, for which the parameter k is 41. It is included in the prefix maximum because $42 \leq 42$. The final assertion selects child 42 in the definition of C and applies the forest child inequality. □

The join in (8) is intentional. Although both hierarchies are strong, a comparison between them cannot be obtained just by substituting a larger oracle into every raw program. Including the earlier candidate as an exact accessible oracle gives a short certified improvement without that invalid assumption. The final entry also reflects over the whole joined language; it is not merely the maximum of two named integers.

8 Limits of the strength claims

8.1 Raw trees are not canonical ordinals

If one interprets Z as zero, S as successor, and L as a countable supremum, then Z and $\mathsf{L}(i \mapsto \mathsf{Z})$ have the same ordinal value. Nevertheless, the reflected hierarchy satisfies

$$H_{\mathsf{Z}}(n) < H_{\mathsf{L}(i \mapsto \mathsf{Z})}(n).$$

It therefore does not descend through this ordinal interpretation. This is an explicit counterexample to rank invariance, not just a missing proof.

Consequently, we do not label Θ as Γ_0, the small Veblen ordinal, or the Bachmann–Howard ordinal, and we do not identify H with a standard fast-growing hierarchy. The raw construction is fully specified and its inequalities are proved directly. Type-theoretic ordinal research distinguishes several tree and order presentations precisely because their constructive operations and equality principles differ [3].

8.2 Oracle order and input order

Neither

$$f \le g \implies \mathcal{R}(f) \le \mathcal{R}(g) \qquad \text{nor} \qquad n \le m \implies \mathcal{R}(f)(n) \le \mathcal{R}(f)(m)$$

is used. An individual interpreted program can decrease when an oracle value or input increases. Enlarging the finite set of programs does not by itself prove either desired inequality when the interpretation of every program changes at the same time. The delivered proofs use only exact embeddings, explicit prefix maxima, positivity, and structural recursion.

This distinction also explains why the unary limit in (5) has a weaker language-level interpretation than the forest limit in (7). The former majorizes each child numerically; the latter provides every child as a callable function. The paper keeps these statements separate.

8.3 A function theorem does not rank every fixed integer

Eventual domination establishes a meaningful separation between functions. It does not assert that our value at 42 exceeds every value at 42 of every definable function: there are arbitrarily long finite programs for arbitrarily large constants. At the selected input, the direct maximum certificate covers the specified depth bound 45. The inequalities over C_5, the intermediate candidate, and the displayed reflection iterates are separate, finite-input theorems.

8.4 Constructive totality and the absence of Busy Beaver search

For each finite syntax bound the enumerated list is finite, and every entry has a total interpreter by construction. We do not enumerate arbitrary Turing machines and wait to discover which ones halt. No undecidable predicate selects a subset of programs. No maximum over all Lean definitions is taken. All higher-type functions used in the closed entry are themselves given by executable definitions.

The construction can still be computationally infeasible by an immense margin. Constructive existence does not imply feasible expansion. Even a very small syntax enumeration grows rapidly. If A_d counts tree-language types, O_d original terms, and T_d tree-language terms, the exact list construction gives

$$A_0 = T_0 = 0, \qquad A_{d+1} = 2 + A_d^2,$$

$$T_{d+1} = O_{d+1} + d + A_d^2 T_d + T_d^2 + 7 + 2A_d.$$

These counts describe syntax, before any enormous semantic evaluation is attempted. The comparison proofs use membership and inequalities rather than materializing those lists at the contest bound.

9 Formal verification and reproducibility

9.1 Axiom discipline

Lean's standard ecosystem includes principles such as propositional extensionality, quotient soundness, and classical choice; it also exposes a command reporting the axioms on which a declaration depends [4]. The contest's rule is implemented here by requiring an empty report, including for the main inequality. This is stricter than merely avoiding a user-written `axiom` declaration.

Some convenient library lemmas may carry stronger dependency sets than their elementary statements suggest. We therefore supply small structural proofs of the needed maximum and

list lemmas. The resulting certificate does not use function extensionality, classical choice, quotient soundness, `sorryAx`, or compiler-trusting native decision procedures. An empty axiom list does not prove Lean's kernel soundness inside Lean; the usual trust in the checker and its foundational rules remains.

The theorem name follows the original contest convention:

```
theorem contender_5_lt_contender_6 :
    Original.contender5 < contender_6 := ...

#print axioms contender_6
#print axioms contender_5_lt_contender_6
```

The archive includes the complete proof in place of the schematic ellipsis. Its audit output is retained separately. The displayed snippet explains what to inspect; it is not a replacement for the checked file.

9.2 Why typechecking remains short

A checker needs to verify that a recursive definition is well formed and that its recursive calls are justified. It does not normally normalize the defined natural number. Lean compiles structural recursion into recursors; the relevant mechanism is documented in its reference manual [5].

There is, however, an engineering hazard. A proof that rewrites a huge specialized closed expression can accidentally make the elaborator unfold the tree schedule or start reducing a semantic computation. The proof package avoids that hazard by proving the needed result for an abstract tree and an abstract reflector first, then specializing the theorem. The abstract proof has no reason to inspect the internal structure of Θ.

This is ordinary abstraction in a proof, not an opaque axiom. Every helper definition and proof is included and checked from source. The timed build regenerates the project's compiled modules in dependency order; only the standard Lean distribution is reused precompiled.

9.3 Observed build and regression checks

The complete source build passed with Lean 4.19.0. Rebuilding all eight modules in the final entry's dependency graph, including their proofs, took **2.962 seconds**. Rebuilding all eleven delivered mathematical modules, including the companion reflection module, smoke checks, and the full axiom audit, took **4.247 seconds** wall-clock time. Both are below the original 15-second definition target on this host; the entire check is also below the one-minute proof target. These are observed wall-clock timings, not hardware-independent guarantees.

Source module	Seconds
`Original.lean`	0.706
`Trees.lean`	0.262
`Reflection.lean`	0.336
`Hierarchy.lean`	0.261
`Arboreal.lean`	0.884
`Forest.lean`	0.381
`Preliminary.lean`	0.158
`ForestHierarchy.lean`	0.161
`Contender.lean`	0.150
`Smoke.lean`	0.318
`Audit.lean`	0.629

All **288 explicitly declared definitions, inductive types, and theorems** in the delivered mathematical and smoke-test modules were audited; **all 288 axiom dependency lists were empty**. A separate traversal of the elaborated dependency graph found 502 reachable declarations, including 169 from this package. Neither the original `contender5` constant nor the intermediate named number appears in the final number's definition closure. There are no axioms, unsafe or partial declarations, or package implementation overrides in that closure. The ordinary core tags for `Nat.modCore` and its helper, and the core representation optimization for `Nat.repr`, are reported explicitly in the audit output; they are not package additions or proof-certification assumptions.

The independent Python/Lean regression comparison passed for **25,134 original term values** in enumeration order:

Depth	Types	Terms	Maximum
0	0	0	0
1	1	2	0
2	2	10	1
3	5	146	2
4	26	24,976	3

The model comparison and the declaration-closure inspection are supplementary validation. The numerical comparison theorem itself is checked by Lean's kernel. Neither large contender was evaluated.

The smoke checks exercise the numerical and higher-type semantics rather than attempting the contest integer. They cover original reverse lookup, out-of-range variables, binder order, malformed application, mismatching casts, closed legacy nodes under a new environment, and tree recursion with natural-number, function, and tree-valued results. The latter includes a countably branching input. Forest checks also select a child index beyond the prefix maximum, pass a bound variable to the family primitive, and call the family from inside successor and limit branches of tree recursion. These are kernel reduction checks at small instances, not a substitute for the general theorems.

The complete source-level audit and reproduction instructions are supplied alongside the article. The recorded runtime uses the Lean 4.19.0 release kernel and standard libraries. In this execution environment, a small application-path launcher is needed because the usual executable-path query is unavailable. Its source is included for transparency; it changes application-path discovery, not the Lean kernel or elaborator. A normal Lean 4.19.0 installation uses the provided build commands directly without this environment-specific launcher.

10 Further research questions

The following directions aim at stronger mathematical information or more economical certified entries. They are proposed research, not claims proved by this archive.

1. **Quantitative translations between object languages.** Construct verified syntax translations from ordinary typed System T, tree-recursive languages, and their forest extensions, with explicit depth overhead. A logical relation at every finite type can avoid assuming function extensionality. Such translations would turn informal comparisons of language expressivity into exact input thresholds.
2. **Ordinal calibration on a restricted tree sublanguage.** Choose canonical trees with increasing cofinal sequences and prove the arithmetic and fixed-point invariants they need. Determine which restricted schedules admit a faithful ordinal interpretation. The raw

reflection hierarchy itself cannot simply be quotiented by ordinal rank, as the counterexample in Section 8 shows.

3. **Lower bounds in established hierarchies.** Prove explicit comparisons with familiar fast-growing functions or normalization bounds. The aim should be certified inequalities with stated fundamental sequences and numerical thresholds, rather than attaching a large ordinal name to a program.
4. **A general theorem for accessible oracle forests.** Abstract the family-language construction over a syntactic category and prove preservation of exact oracle access under arbitrary finite composition. The present implementation supplies one concrete total language and the key diagonal inequality; a general metatheorem could organize many further contests.
5. **More expressive, still total object languages.** Add further strictly positive inductive types or carefully stratified dependent type formers, each with a verified interpreter. A new language must remain below its meta-language in the relevant sense; unrestricted self-interpretation cannot be inserted by assertion.
6. **Uniform reflection over finite universe fragments.** Investigate a sequence of syntactic universes with explicit interpretation maps and verified consistency boundaries. This may give a stronger systematic advance than adding further successor padding. It is also substantially harder than the present dependent finite-type uniformization.
7. **Semantic preservation of normalization optimizations.** Replace explicit Cartesian-product enumeration by sharing, dynamic programming, or canonical syntax DAGs, while proving that the numerical maximum is unchanged. The objective is useful small-instance experimentation and a cleaner implementation, not feasible evaluation of the final entry.
8. **An official, reproducible Lean contest protocol.** Fix the toolchain, machine or normalized benchmark, byte accounting, allowed imports, axiom policy, and treatment of compiled dependency caches. Separate definition checking from normalization explicitly. This would make future claims of admissibility substantially easier to compare.
9. **A verified source bridge for the old contender.** Formalize an intermediate syntax/semantics specification that can be checked from both Coq and Lean, or establish a verified translation for this fragment. That would remove the manual cross-language source audit from the baseline identity argument.
10. **Economy of the final construction.** Search for a shorter entry with the same or stronger certified lower bounds, and for stronger entries with comparable check time. Optimizing a precisely defined language and checker budget is a concrete experimental problem; there is no evidence here of a globally optimal submission.

11 Conclusion

The final definition is a concrete constructive natural number with a checked strict comparison against the repository's fifth contender. Its main advance is the combination of total higher-type interpretation, explicit access to forests of earlier evaluators, and a dependent family of structural schedules. The exact finite-depth diagonal theorem explains what each reflection step achieves. The separate fixed-input certificates explain why the chosen entry wins the required comparison.

The strongest claims in the paper are backed by Lean proofs, while the remaining boundaries are explicit: source-faithful cross-language porting, machine-specific timing, raw trees rather than canonical ordinals, and no universal evaluation of all Lean programs. The result is a reproducible entry with explicit, checked strength claims.

A Guide to the source modules

Module	Role
`Original.lean`	Exact computational port; original enumeration, maximum invariant, membership characterizations.
`Reflection.lean`	Unary reflection for the original natural-number language; a smaller independently checked diagonalization module.
`Trees.lean`	Raw tree type, arithmetic, functional type family, uniformization schedule, and auxiliary fast-tree lemmas.
`Arboreal.lean`	Full tree-recursive object language and unary reflector $\mathcal{R}$.
`Hierarchy.lean`	Generic tree-indexed unary reflection, prefix maximum, padding, and path-growth theorems.
`Forest.lean`	Accessible countable oracle family and forest reflector $\mathcal{F}$.
`Preliminary.lean`	The completed unary candidate family, its strict source comparison, and its 43-reflection lower bound.
`ForestHierarchy.lean`	Structural stages with exact access to their entire child families and corresponding growth certificates.
`Contender.lean`	The final joined oracle forest, natural number, and fixed-input comparison certificates.
`Smoke.lean`	Small exact semantic regression checks.
`Audit.lean`	Explicit axiom dependency checks of the deliverable.

The reproduction scripts and the README state the exact build order. Source proofs, numerical definitions, and verification logs are all included. The upstream MIT license is retained with the source-faithful port and the original files.

References

[1] Cody Roux and contributors. *name-the-biggest-number*: README and `Contender.v`. Repository revision `d56dce013e6264292533efdc5e1de71863c6fb86`. https://github.com/codyroux/name-the-biggest-number. Accessed 12 October 2026. The original Contender 5 is credited in the source to Samuel Grütter.

[2] Martín Escardó. *Ordinals in Goedel's system T and in Martin-Loef Type Theory*. 2010; Agda development, November 2011. https://martinescardo.github.io/papers/ordinals/ordinals.html.

[3] Nicolai Kraus, Fredrik Nordvall Forsberg, and Chuangjie Xu. *Type-Theoretic Approaches to Ordinals*. arXiv:2208.03844. https://arxiv.org/abs/2208.03844.

[4] The Lean community. *Theorem Proving in Lean 4*, "Axioms and Computation." https://lean-lang.org/theorem_proving_in_lean4/Axioms-and-Computation/.

[5] The Lean community. *Lean Language Reference*, version 4.19.0, "Recursive Definitions." https://lean-lang.org/doc/reference/4.19.0/Definitions/Recursive-Definitions/.